

Versão: 1.0

Última modificação: 03/05/25

Tutorial OAuth 2.0 do Google

OAuth 2.0 é um protocolo de autorização que permite que um aplicativo de terceiros (como um site ou app) acesse dados de um usuário armazenados no Google (como Gmail, Google Drive, etc.) sem precisar da senha do usuário.

Resumo:

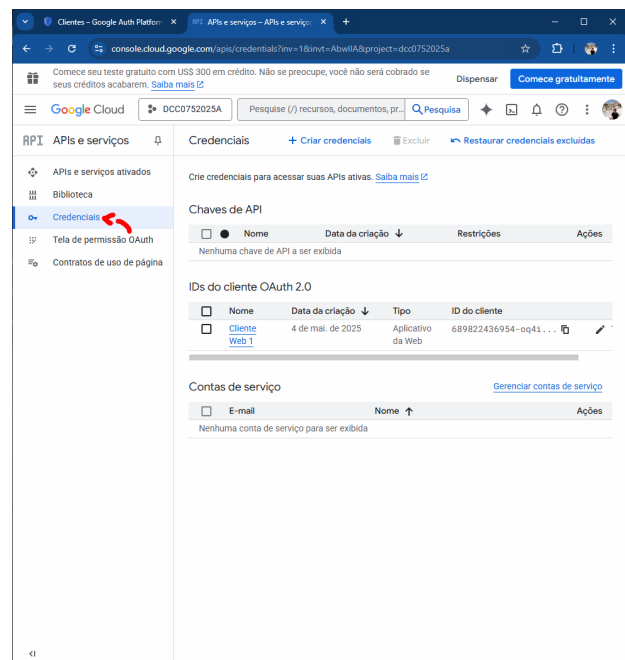
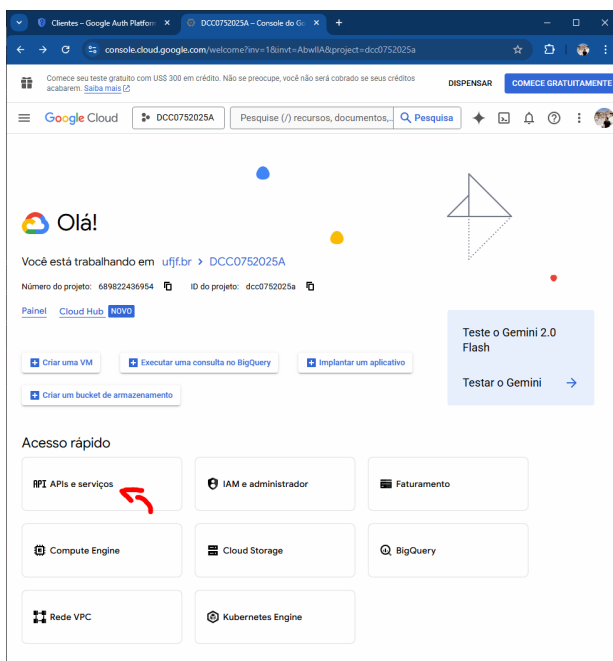
1. O usuário acessa seu aplicativo e escolhe fazer login com a conta do Google.
2. O aplicativo redireciona o usuário para uma tela de login do Google (controlada pelo próprio Google).
3. O usuário faz login e o Google pede autorização para que seu app acesse determinados dados (ex: e-mail, nome, Drive, etc.).
4. Se o usuário aceitar, o Google redireciona de volta ao seu app com um "código de autorização".
5. O seu app usa esse código para obter um "token de acesso" do Google.
6. Com esse token, seu app pode acessar os dados permitidos do usuário no Google.
7. Quando o token expira, o app pode usar um "refresh token" para pedir um novo.

Etapa 1: Configurar o Projeto no Google Cloud

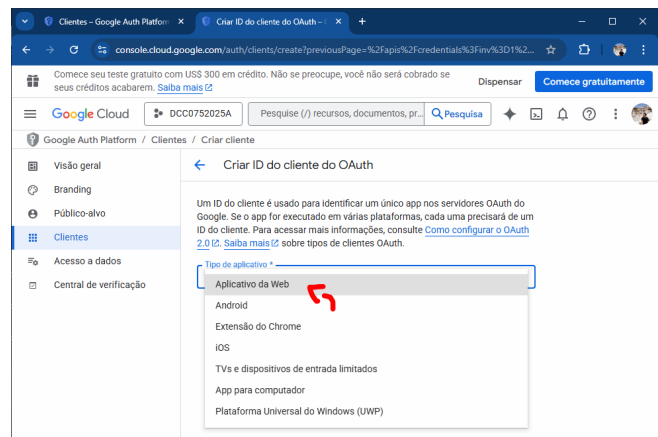
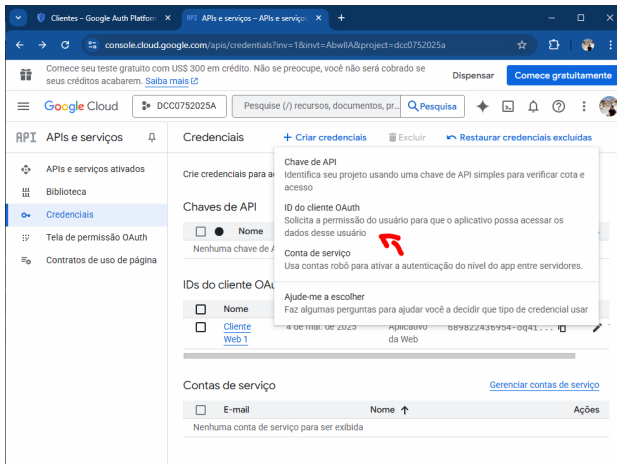
Acesse o Google Cloud Console e crie um novo projeto:

<https://console.cloud.google.com/>

Faça a seguinte configuração conforme as telas abaixo:

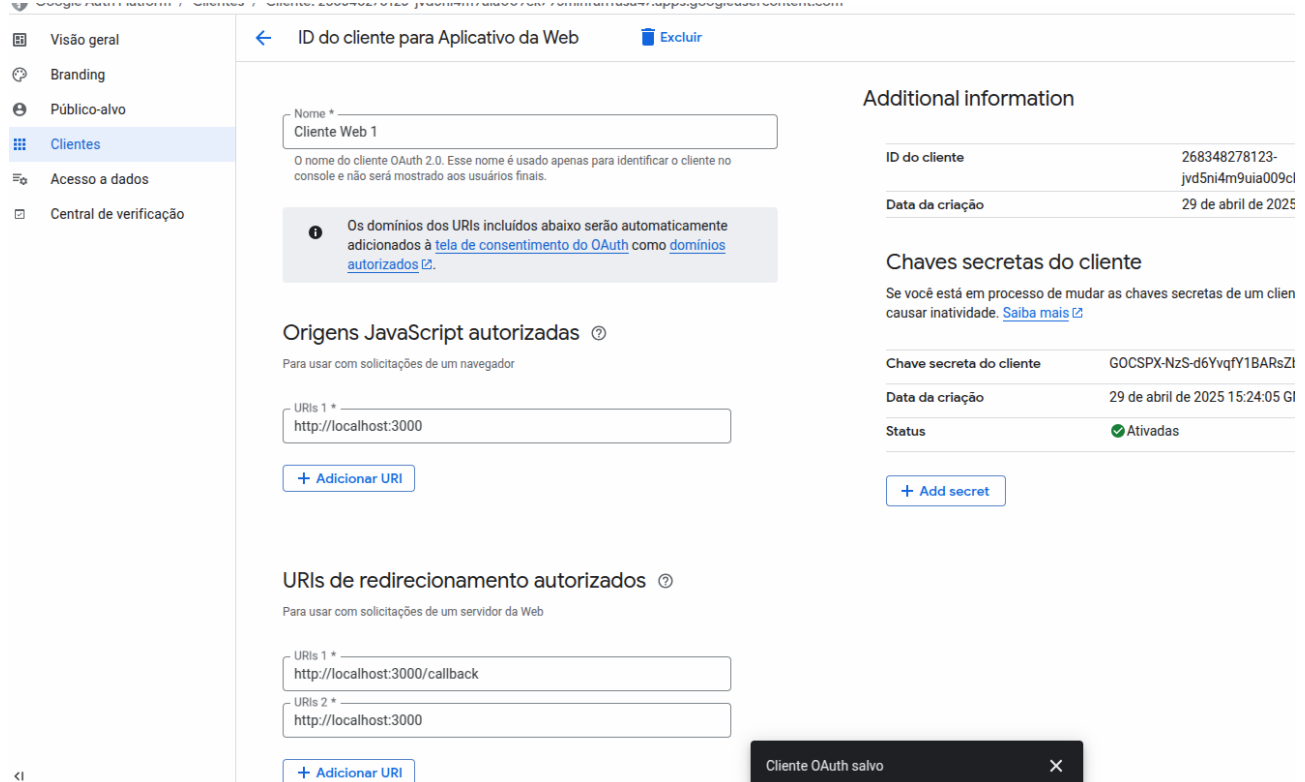


Clicar em APIs e serviços → Credenciais → ID do Cliente Oauth → Aplicativo da Web



Adicionar Origem JavaScript Autorizada e URI de

redirecionamento autorizada:



Nas informações adicionais **ID do Cliente** e **Chave secreta do cliente** serão utilizadas em um arquivo `.env` mais adiante.

Instalação do <https://bun.sh>:

Abra o terminal e adicione o seguinte código:

```
mkdir google-oauth-express
cd google-oauth-express
bun init
-> opção 'blank'
bun add express google-auth-library dotenv
```

Verifique se o comando funciona, ou se há a necessidade de adicioná-lo ao seu ambiente *bash*:

```
sudo apt install source
source /home/user/.bashrc
```

Dentro da pasta de destino deve ser respeitada a seguinte estrutura:

~~google-oauth-express/~~

```
|— public/
|   |— index.html
|— src/
|   |— index.js
|— .env
|— bun.lockb
```

Crie um arquivo *process.env* na raiz do projeto com o seguinte conteúdo:

```
GOOGLE_CLIENT_ID=seu_client_id
GOOGLE_CLIENT_SECRET=seu_client_secret
GOOGLE_REDIRECT_URI=http://localhost:3000/callback
```

Etapa 3: Implementar o Código de src/index.js (o caminho depende da organização do seu projeto):

```
import express from 'express';
import { OAuth2Client } from 'google-auth-library';
import dotenv from 'dotenv';

dotenv.config();

const app = express();
const PORT = 3000;

const client = new OAuth2Client(
  process.env.GOOGLE_CLIENT_ID,
  process.env.GOOGLE_CLIENT_SECRET,
  process.env.GOOGLE_REDIRECT_URI
);

app.get('/', (req, res) => {
  const url = client.generateAuthUrl({
    access_type: 'offline',
    scope: ['email', 'profile', 'openid'],
  });
  res.send(`<a href="${url}">Login com Google</a>`);
});

app.get('/callback', async (req, res) => {
  try {
    const { code } = req.query;
    const { tokens } = await client.getToken(code);
    client.setCredentials(tokens);

    const ticket = await client.verifyIdToken({
      idToken: tokens.id_token,
      audience: process.env.GOOGLE_CLIENT_ID,
    });

    const payload = ticket.getPayload();
    const name = encodeURIComponent(payload.name);
    const email = encodeURIComponent(payload.email);

    // Redireciona para a página de boas-vindas com parâmetros
    res.redirect(`/welcome?name=${name}&email=${email}`);
  } catch (error) {
    console.error('Erro na autenticação:', error);
    res.status(500).send('Erro ao autenticar com o Google.');
```

```
  }
});

app.get('/welcome', (req, res) => {
  const { name, email } = req.query;
```

```
res.send(`
  <h1>Bem-vindo, ${name}!</h1>
  <p>Seu e-mail: ${email}</p>
`);
});

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

public/index.html:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <title>Login com Google</title>
</head>
<body>
  <h1>Autenticação com Google OAuth2</h1>
  <a href="/">Clique aqui para fazer login</a>
</body>
</html>
```

Inicie o Servidor:

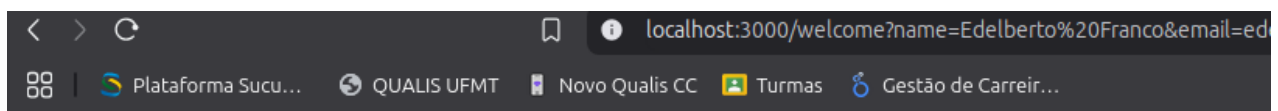
Abra o terminal e digite o comando

```
bun --env-file=.env src/index.js
```

Vá para <http://localhost:3000>

Clique no link para iniciar o processo de login com o Google.

Após a autenticação, você será redirecionado de volta e verá uma mensagem de boas-vindas com seu nome.



Bem-vindo, Edelberto Franco!

Seu e-mail: edelberto@gmail.com

APÊNDICE

O token de acesso (access token) não é exibido diretamente no navegador por padrão quando você usa OAuth 2.0 com um backend como Node.js. Ele está no fluxo que você implementou (authorization code flow). Para vê-lo no terminal, atualize a entrada “app.get('/callback’”.

```
app.get('/callback', async (req, res) => {
  const { code } = req.query;
  const { tokens } = await client.getToken(code);
  console.log('Tokens recebidos do Google:', tokens); // print no
terminal do token
  client.setCredentials(tokens);

  const ticket = await client.verifyIdToken({
    idToken: tokens.id_token,
    audience: process.env.GOOGLE_CLIENT_ID,
  });
  const payload = ticket.getPayload();
  res.send(`Olá, ${payload.name}!`);
});
```